

JavaScript Global Execution Context - Complete Guide

What is Global Execution Context (GEC)?

- The **Global Execution Context** is the default execution context in JavaScript. It's the base/foundational context that gets created when JavaScript code starts executing. Every JavaScript program begins with the Global Execution Context being created and pushed onto the call stack.
- When we give js code to the browser, JS Engine will allocate(create) a global memory block for the execution of javascript code, called Global Execution Context.
- Here, we have a window variable which has reference of Global Execution Context.

Window variable

- window variable or window object -> everything is object in js.
- Window is a global variable which stores the reference of Global Execution Context.
- Window object is also known as Global Object because it is available anywhere in the program.
- Window object has pre-defined state and behaviour.
- Variable declared with var always goes to global scope and can be accessible by window object.
- Any variable created in global scope will be added to Window object implicitly by JS Engine.

Key Characteristics:

- **Created automatically** when JavaScript engine starts executing code
- **Only one GEC** exists per JavaScript program/file
- **Base context** for all other execution contexts
- **Contains global variables and functions**
- **Provides the global object** (`window` in browsers, `global` in Node.js)

Global Execution Context Creation Process

The GEC is created in **two phases**:

Phase 1: Variable Phase (Creation Phase/Memory Allocation Phase)

During this phase, JavaScript engine:

1. **Creates the global object** (`window` in browsers)
2. **Sets up the `this` keyword** (points to global object)
3. **Scans the entire code** for variable and function declarations
4. **Allocates memory** for variables and functions
5. **Initializes variables** with `undefined`
6. **Stores complete function definitions** in memory

```
console.log("Start");
console.log(a); // undefined (not error due to hoisting)
console.log(b); // undefined

var a = 10;
let b = 20;
const c = 30;

function greet() {
  console.log("Hello World");
}

console.log("End");
```

Memory allocation during Variable Phase:

```
Global Memory:
├─ a: undefined (var - hoisted)
├─ b: undefined (let - in TDZ)
├─ c: undefined (const - in TDZ)
├─ greet: function greet() { console.log("Hello World"); }
├─ this: window object
└─ window: global object
```

Phase 2: Execution Phase (Code Execution Phase)

During this phase, JavaScript engine:

1. **Executes code line by line**
2. **Assigns actual values** to variables
3. **Calls functions** when encountered
4. **Resolves variable references**

```
// Execution Phase starts
console.log("Start");           // Executes immediately
console.log(a);                 // Logs: undefined (from memory)
console.log(b);                 // ReferenceError: Cannot access 'b' before
initialization

var a = 10;                     // a gets value 10
let b = 20;                     // b gets value 20, exits TDZ
const c = 30;                   // c gets value 30, exits TDZ

function greet() {              // Already in memory, skipped
  console.log("Hello World");
}

console.log("End");             // Executes
```

Hoisting in Global Execution Context

Variable Hoisting

```
// This code:
console.log(x); // undefined
console.log(y); // ReferenceError
console.log(z); // ReferenceError

var x = 5;
let y = 10;
const z = 15;

// Is interpreted as:
var x; // hoisted and initialized with undefined
// let y; // hoisted but in TDZ
// const z; // hoisted but in TDZ

console.log(x); // undefined
console.log(y); // ReferenceError: Cannot access 'y' before initialization
console.log(z); // ReferenceError: Cannot access 'z' before initialization

x = 5;
y = 10;
z = 15;
```

Function Hoisting

```
// Function called before declaration - works due to hoisting
console.log("Start");
greet("Avinash"); // Works perfectly

function greet(name) {
  console.log(`Hello ${name}`);
}
console.log("End");

// Output:
// Start
// Hello Avinash
// End
```

Function Expression Hoisting

```
console.log(myFunc); // undefined (variable is hoisted, function is not)
myFunc(); // TypeError: myFunc is not a function

var myFunc = function() {
  console.log("I'm a function expression");
};
```

Temporal Dead Zone (TDZ)

Variables declared with `let` and `const` are hoisted but remain in the **Temporal Dead Zone** until their declaration is reached.

```
console.log("Start");

// TDZ for 'a' starts here
console.log(a); // ReferenceError: Cannot access 'a' before initialization

let a = 10; // TDZ for 'a' ends here
console.log(a); // 10

// TDZ for 'b' starts here
console.log(b); // ReferenceError: Cannot access 'b' before initialization

const b = 20; // TDZ for 'b' ends here
console.log(b); // 20
```

TDZ Example with Blocks

```
var a = 10;

{
  console.log(a); // ReferenceError: Cannot access 'a' before initialization
  let a = 20; // This 'a' shadows the global 'a'
}
```

Global Object and Scope

Browser Environment (**window** object)

```
var globalVar = "I'm global";
let blockScoped = "I'm block scoped";
const constant = "I'm constant";

function globalFunction() {
  console.log("I'm a global function");
}

// Accessing through global object
console.log(window.globalVar); // "I'm global"
console.log(window.blockScoped); // undefined (let doesn't attach to window)
console.log(window.constant); // undefined (const doesn't attach to window)
console.log(window.globalFunction); // function globalFunction() {...}

// Direct access (same as above for var and function declarations)
console.log(globalVar); // "I'm global"
console.log(globalFunction); // function globalFunction() {...}
```

Auto-Global Variables

```
function createAutoGlobal() {
  autoGlobal = "I become global"; // No var/let/const declaration
  var localVar = "I'm local";
}

createAutoGlobal();

console.log(autoGlobal); // "I become global" (accessible globally)
console.log(localVar); // ReferenceError: localVar is not defined
```

Scope Chain in Global Context

```

var globalVar = "Global";

function outerFunction() {
  var outerVar = "Outer";

  function innerFunction() {
    var innerVar = "Inner";

    // Scope chain: innerFunction → outerFunction → Global
    console.log(innerVar); // "Inner" (own scope)
    console.log(outerVar); // "Outer" (parent scope)
    console.log(globalVar); // "Global" (global scope)
  }

  innerFunction();
}

outerFunction();

```

Memory Management in GEC

Variable Storage Locations

```

// Global Execution Context Memory Layout

// Script Scope (let/const)
let scriptVar = "In script scope";
const scriptConst = "In script scope";

// Global Scope (var/function)
var globalVar = "In global scope";
function globalFunc() {}

// Auto-global (no declaration)
autoGlobalVar = "In global scope";

```

Memory Organization:

```

Call Stack:
└─ Global Execution Context
    └─ Global Scope (window object)
        ├── globalVar: "In global scope"
        ├── globalFunc: function
        └─ autoGlobalVar: "In global scope"
    └─ Script Scope
        ├── scriptVar: "In script scope"
        └─ scriptConst: "In script scope"
    └─ this: window object
  
```

Variable Conflicts and Shadowing

Global vs Local Scope

```

var name = "Global Name";

function testScope() {
    var name = "Local Name"; // Shadows global variable
    console.log(name); // "Local Name"
}

testScope();
console.log(name); // "Global Name"
  
```

TDZ Conflicts

```

b = 20;           // Assignment before declaration
let b = 200;      // ReferenceError: Cannot access 'b' before initialization
console.log(b);

// This happens because:
// 1. In variable phase, 'b' is hoisted but placed in TDZ
// 2. In execution phase, assignment tries to access 'b' before declaration
// 3. ReferenceError is thrown due to TDZ violation
  
```

Block Scope in Global Context

```

console.log("Start");

var a = 10;

{
  console.log("Inside a local block");
  b = 20; // This will cause an error in strict mode
  console.log(a); // 10 (can access global 'a')
  console.log(b); // ReferenceError if 'let b = 2;' is declared below
}

console.log("Outside a local block");
let b = 2; // This creates TDZ for entire scope, including the block above
console.log(a); // 10
console.log(b); // 2

```

Call Stack and Global Execution Context

Call Stack Visualization

```

function third() {
  console.log("Third function");
}

function second() {
  console.log("Second function");
  third();
}

function first() {
  console.log("First function");
  second();
}

first();

```

Call Stack Evolution:

Step 1: Program starts
[Global Execution Context]

Step 2: first() is called
[first() Execution Context]
[Global Execution Context]

Step 3: second() is called from first()
[second() Execution Context]
[first() Execution Context]
[Global Execution Context]

Step 4: third() is called from second()
[third() Execution Context]
[second() Execution Context]
[first() Execution Context]
[Global Execution Context]

Step 5: third() completes and is popped
[second() Execution Context]
[first() Execution Context]
[Global Execution Context]

Step 6: second() completes and is popped
[first() Execution Context]
[Global Execution Context]

Step 7: first() completes and is popped
[Global Execution Context]

Step 8: Program ends, GEC remains until page/process ends

Function Execution Context vs Global Execution Context

Function Execution Context Creation

```
console.log("Start");

function sum(a, b) {
  console.log("Inside sum function");
  var result = a + b;
  console.log("Sum:", result);
  return result;
}

const x = sum(10, 20);
console.log("x =", x);
console.log("End");
```

Function Execution Context Process:

Variable Phase:

```
Function Execution Context for sum():
├─ Parameters: a = undefined, b = undefined
├─ Local variables: result = undefined
├─ this: window object (in non-strict mode)
└─ return value: undefined
```

Execution Phase:

```
Function Execution Context for sum():
├─ Parameters: a = 10, b = 20 (from arguments)
├─ Local variables: result = 30 (computed)
├─ this: window object
└─ return value: 30 (from return statement)
```

Advanced Global Execution Context Concepts

Multiple Script Tags

```
<!-- Script 1 -->
<script>
  var globalVar1 = "From Script 1";
  function globalFunc1() {
    console.log("Function from Script 1");
  }
</script>

<!-- Script 2 -->
<script>
  console.log(globalVar1); // "From Script 1" (accessible)
  globalFunc1(); // Works perfectly

  var globalVar2 = "From Script 2";
</script>

<!-- Script 3 -->
<script>
  console.log(globalVar1); // "From Script 1"
  console.log(globalVar2); // "From Script 2"
  // All share the same Global Execution Context
</script>
```

Global Execution Context with Modules

```
// module1.js
export const moduleVar = "I'm from module";
export function moduleFunc() {
  console.log("Module function");
}

// main.js
import { moduleVar, moduleFunc } from './module1.js';

// Each module has its own module scope, but shares global objects
console.log(window); // Still accessible
console.log(globalThis); // Available in all environments
```

Error Handling in Global Execution Context

Reference Errors

```
// These will cause ReferenceError
console.log(undeclaredVariable); // ReferenceError: undeclaredVariable is not
defined

// TDZ violations
console.log(letVar); // ReferenceError: Cannot access 'letVar' before initialization
let letVar = 10;

console.log(constVar); // ReferenceError: Cannot access 'constVar' before
initialization
const constVar = 20;
```

Type Errors

```
var notAFunction = "I'm a string";
notAFunction(); // TypeError: notAFunction is not a function

var myFunc = function() {
  console.log("I'm a function");
};

// Before declaration
myFunc(); // TypeError: myFunc is not a function (myFunc is undefined)
```

Global Execution Context Best Practices

1. Minimize Global Variables

```
// Bad: Polluting global scope
var userName = "John";
var userAge = 25;
var userEmail = "john@example.com";

function processUser() {
  // Using global variables
}

// Good: Use namespace or module pattern
const UserApp = {
  data: {
    name: "John",
    age: 25,
    email: "john@example.com"
  },

  processUser: function() {
    // Access through namespace
  }
};
```

2. Use Strict Mode

```
"use strict";

// Prevents auto-global variables
function createVariable() {
  undeclaredVar = "This will throw an error in strict mode";
  // ReferenceError: undeclaredVar is not defined
}
```

3. Proper Variable Declarations

```
// Bad: Multiple var declarations
var a = 1;
var b = 2;
var c = 3;

// Good: Single var declaration
var a = 1,
    b = 2,
    c = 3;

// Better: Use let/const appropriately
const CONFIG = {
  apiUrl: 'https://api.example.com',
  timeout: 5000
};

let currentUser = null;
```

Global Execution Context in Different Environments

Browser Environment

```
// Global object is 'window'
console.log(this === window); // true (in non-strict mode)

var browserVar = "I'm in browser";
console.log(window.browserVar); // "I'm in browser"

// Browser-specific globals
console.log(document); // DOM document object
console.log(navigator); // Browser information
console.log(location); // Current URL information
```

Node.js Environment

```
// Global object is 'global'
console.log(this === global); // false (in module scope)
console.log(global); // Global object

// Node.js specific globals
console.log(__dirname); // Current directory
console.log(__filename); // Current file
console.log(process); // Process object
console.log(require); // Module loader
```

Web Workers

```
// Global object is 'self'
console.log(this === self); // true
console.log(self); // Worker global scope

// No access to DOM
console.log(typeof document); // "undefined"
console.log(typeof window); // "undefined"
```

Debugging Global Execution Context

Using Browser DevTools

```
// Set breakpoints and inspect global scope
function debugGlobalScope() {
  debugger; // Execution will pause here

  // In DevTools Console, you can:
  // - Type 'window' to see all global variables
  // - Type 'this' to see global object
  // - Inspect scope in Sources tab
}

debugGlobalScope();
```

Memory Usage Analysis

```
// Monitor global variables
function analyzeGlobalMemory() {
  const globalKeys = Object.keys(window);
  console.log("Global variables count:", globalKeys.length);

  // Find custom globals (filter out built-in properties)
  const customGlobals = globalKeys.filter(key => {
    return !key.startsWith('webkit') &&
      !key.startsWith('chrome') &&
      typeof window[key] !== 'function' ||
      key.match(/^[a-z]/); // Custom naming convention
  });

  console.log("Custom global variables:", customGlobals);
}
```

Performance Implications

Global Variable Access Speed

```

var globalVar = "Global";

function performanceTest() {
  var localVar = "Local";

  console.time("Global Access");
  for (let i = 0; i < 1000000; i++) {
    let temp = globalVar; // Slower - scope chain lookup
  }
  console.timeEnd("Global Access");

  console.time("Local Access");
  for (let i = 0; i < 1000000; i++) {
    let temp = localVar; // Faster - direct access
  }
  console.timeEnd("Local Access");
}

performanceTest();

```

Memory Leaks Prevention

```

// Bad: Global variables never get garbage collected
var globalData = new Array(1000000).fill("data");

function processData() {
  // Using global data
  return globalData.map(item => item.toUpperCase());
}

// Good: Use local scope and clean up
function processDataBetter() {
  const localData = new Array(1000000).fill("data");
  const result = localData.map(item => item.toUpperCase());

  // localData will be garbage collected after function ends
  return result;
}

```

Common Global Execution Context Pitfalls

1. Variable Hoisting Confusion

```
// Unexpected behavior due to hoisting
var a = 1;

function hoistingPitfall() {
  console.log(a); // undefined (not 1!)

  if (false) {
    var a = 2; // This declaration is hoisted to top of function
  }
}

hoistingPitfall();
```

2. Accidental Global Creation

```
function accidentalGlobal() {
  var localVar = "I'm local";
  accidentalVar = "I'm accidentally global"; // Missing var/let/const
}

accidentalGlobal();
console.log(accidentalVar); // "I'm accidentally global" - Oops!
```

3. TDZ Timing Issues

```
// This will throw an error
console.log(myLet); // ReferenceError

if (true) {
  let myLet = "I'm in TDZ until this line";
}
```

Summary

The Global Execution Context is the foundation of JavaScript execution. Understanding how it works - from the variable and execution phases to hoisting, scope chains, and memory management - is crucial for writing efficient JavaScript code and debugging issues effectively.

Key takeaways:

- **GEC is created automatically** when JavaScript starts executing
- **Two phases:** Variable phase (memory allocation) and Execution phase (code execution)
- **Hoisting** allows access to variables and functions before their declaration
- **TDZ** prevents access to `let/const` variables before initialization

- **Global variables** should be minimized for better performance and maintainability
- **Scope chain** determines variable resolution order
- **Memory management** in GEC affects application performance

Understanding these concepts enables you to write better JavaScript code, avoid common pitfalls, and debug issues more effectively.

JavaScript Hoisting and Temporal Dead Zone

By Saket Bhatnagar | June 27, 2025 | Beginner to Intermediate

Table of Contents

1. [Hoisting](#)
2. [Temporal Dead Zone \(TDZ\)](#)

Hoisting

Hoisting is the ability of the JavaScript engine to access variables and functions before their declaration line appears in code.

This doesn't mean JavaScript moves the code physically upward - instead, it creates memory blocks during the first phase of code execution.

JavaScript Engine Execution Phases

JavaScript engine executes code in two phases:

1. **Memory Creation Phase or Variable Phase** - JS engine scans the whole file and allocates memory for all variables and functions in the same sequence as they appear in the code. When all memory is allocated, JS engine moves to execution phase.
2. **Execution Phase** - JS executes code line by line and assigns values to variables.

What happens in memory phase:

- Variables declared with `var` are auto-initialized with `undefined`
- Variables declared with `let` and `const` are not initialized with any value in memory phase and remain empty until JS engine reads the declaration statement in execution phase and initialize it with the value
- In case of function, whole function code is stored in memory block (only for functions those are declared with `function` keyword)

Example 1: var Hoisting

```
console.log(a); // undefined
var a = 10;
```

Note: During memory phase, `a` is initialized with `undefined`. So in execution phase, when `console.log(a)` runs, JS finds that memory block and found value `undefined` and prints `undefined`.

Example 2: let Hoisting (TDZ)

```
console.log(b); // ReferenceError
let b = 10;
```

Note: In memory phase, `b` is only allocated memory but not initialized means memory block is empty. So in execution phase, `console.log(b)` runs, JS finds that memory block but memory block is empty and that's why it throws error because we are trying to access the memory block but nothing is stored there to be printed or logged.

Example 3: Function Hoisting

```
sayHi(); // Output: Hi

function sayHi() {
  console.log('Hi');
}
```

Note: In memory phase, `sayHi` is stored with its code. So in execution phase, when `sayHi()` runs, JS finds that memory block and found function code and calls it.

Function declarations are fully hoisted with their code in memory phase. That's why you can call `sayHi()` before its declaration.

Interview Key Point: Hoisting = memory is ready before code runs. JS doesn't move the code; it just prepares memory in advance.

Temporal Dead Zone (TDZ)

It is the time frame between memory allocation and initialization, in this time frame we cannot access the variable. If we try to access memory block (variable) before it is initialized, as JS engine is not able to find any value to be printed or logged, JS engine throws error and this is called **Temporal Dead Zone (TDZ)**.

Why we call dead zone? Because we are not able to access the variable before it is initialized.

Example 1: let (TDZ)

```
console.log(b); // ReferenceError  
let b = 10;
```

Note: In memory phase, `b` is only allocated memory but not initialized means memory block is empty. So in execution phase, `console.log(b)` runs, JS finds that memory block but memory block is empty and that's why it throws error because we are trying to access the memory block but nothing is stored there to be printed or logged.

Example 2: const (TDZ)

```
console.log(b); // ReferenceError  
const b = 10;
```

Note: In memory phase, `b` is only allocated memory but not initialized means memory block is empty. So in execution phase, `console.log(b)` runs, JS finds that memory block but memory block is empty and that's why it throws error because we are trying to access the memory block but nothing is stored there to be printed or logged.